

Estudo e Implementação de Árvores B

Leonardo da S. Simões

Departamento de Engenharia de Computação - Centro Federal de Educação Tecnologia de Minas Gerais, Leopoldina (CEFET-MG) - Brasil

noxem0505@gmail.com

Abstract. This paper presents an analysis of the use of B-trees. The objective is to understand how this data structure works and contributes to the efficiency of record search, insertion, and deletion operations. To this end, a literature review was conducted based on classic works in the field, and a version of the structure was implemented in C++. The results demonstrate that B-trees significantly reduce the number of disk accesses when compared to traditional structures. It is concluded that their use is essential in applications that handle large volumes of data.

Resumo. Este trabalho apresenta uma análise sobre o uso de árvores B. O objetivo é compreender como essa estrutura funciona e contribui para a eficiência das operações de busca, inserção e remoção de registros. Para isso, foi realizada uma revisão bibliográfica baseada em obras clássicas da área e implementada uma versão da estrutura em linguagem C++. Os resultados demonstram que as árvores B reduzem significativamente o número de acessos ao disco quando comparadas a estruturas tradicionais. Conclui-se que seu uso é fundamental em aplicações que manipulam grandes volumes de dados.

Introdução

A árvore B é uma estrutura de dados do tipo árvore auto-balanceada, projetada para funcionar de forma eficiente em dispositivos de memória secundária, como discos rígidos e SSDs. Ela permite realizar operações de busca, inserção e remoção de chaves em tempo logarítmico e é amplamente utilizada em sistemas que manipulam grandes volumes de dados, como bancos de dados e sistemas de arquivos. Sua principal característica é reduzir a quantidade de acessos ao disco, melhorando o desempenho das operações sobre os dados armazenados.

Ela foi criada porque estruturas tradicionais de árvore de busca não são adequadas quando os dados estão armazenados em memória secundária (HDs e SSDs).

O problema é que acessar um disco é muito mais lento do que acessar a memória RAM. Se utilizássemos uma árvore binária comum para armazenar milhões de registros em disco, a altura da árvore seria relativamente grande, exigindo muitos acessos ao disco durante uma busca. Cada acesso adicional aumenta significativamente o tempo de execução.

A árvore B foi projetada para resolver esse problema. Ela permite que cada nó

armazene várias chaves e tenha muitos filhos, reduzindo drasticamente a altura da árvore. Como consequência, são necessários menos acessos ao disco para encontrar uma informação.

Como uma Árvore B funciona

Os nós de uma Árvore B são geralmente projetados para ocupar uma página inteira de disco. Dessa forma, quando uma página é carregada para a memória principal, diversas chaves podem ser examinadas antes que seja necessário realizar um novo acesso ao dispositivo de armazenamento. Essa característica contribui para o aumento da eficiência das operações de busca, inserção e remoção.

Além disso, cada nó pode armazenar várias chaves e possuir um grande número de filhos, característica conhecida como alto fator de ramificação. Quanto maior o fator de ramificação, menor tende a ser a altura da árvore, reduzindo a quantidade de páginas que precisam ser acessadas para localizar uma informação. Consequentemente, a Árvore B consegue manter operações eficientes mesmo quando utilizada para armazenar milhões de registros.

As principais propriedades de uma Árvore B são:

- As chaves em cada nó permanecem ordenadas.
- Os nós internos possuem um número de filhos igual ao número de chaves mais um.
- Todas as folhas encontram-se no mesmo nível, garantindo o balanceamento da estrutura.
- Existe um limite mínimo e máximo de chaves por nó, determinado pelo grau mínimo da árvore.
- A altura da árvore cresce de forma logarítmica em relação ao número de elementos armazenados.

Teorema: Para qualquer árvore B de altura h , $t \geq 2$ e n nós, se $n \geq 1$, então $h \leq \log_t \left(\frac{n+1}{t} \right)$. Sendo t o grau mínimo da árvore, n o número de chaves em um nó.

As operações fundamentais realizadas em uma Árvore B são busca, inserção e remoção.

Busca: A operação de busca em uma árvore B é semelhante à operação em uma árvore binária, a diferença é que enquanto na árvore binária tomamos uma decisão de ramificação de duas vias, na árvore B tomamos a decisão de ramificação de acordo com o número de filhos, mais especificamente uma decisão de $n+1$ (n sendo o número de chaves) vias. A busca toma como parâmetros x e k , x sendo um ponteiro para um nó raiz de uma subárvore e k sendo a chave que vai ser procurada nessa subárvore. A busca deve retornar um par ordenado de (y, i) , que consiste em um nó y e um índice i que indica a i -ésima chave do nó y .

Busca_Arvore_B(x, k)

```
1      i = 1
2      while i <= x.n e k > x.chave[ i - 1]
3          i = i + 1
4      if i <= x.n e k = x.chave[ i - 1]
5          return (x, i)
6      elseif x.folha
7          return NULL
8      else Disk-Read(x.filho[ i - 1])
9      return Busca_Arvore_B(x.filho[ i - 1], k)
```

As linhas 1-3 encontram o menor índice i tal que $k \leq x.chave[i - 1]$ ou definem i como $x.n + 1$. As linhas 4-5 verificam se agora descobrimos a chave e a retornam se a tivermos descoberto. As linhas 6 e 7 verificam se x é uma folha e se sim, retornam um null. As linhas 8 e 9 executam uma recursão para procurar a subárvore adequada de x , após executar a necessária operação DISK-READ naquele filho. Os nós encontrados durante a recursão formam um caminho simples descendente desde a raiz da árvore. Portanto, o procedimento Busca_Arvore_B acessa $O(h) = O(\log t)$ páginas de disco, onde h é a altura da árvore B e n é o número de chaves na árvore B . Já que $x.n$ é no máximo $2t - 1$, ou seja, menor que $2t$, o while das linhas 2 e 3 tem complexidade de tempo $O(t)$, no pior caso a chave que queremos achar estará em uma folha, então o Busca_Arvore_B rodará esse while em cada nó da raiz até a folha, logo o tempo total será de $O(th) = O(t \log t)$, ou seja logaritmo, já que t é constante.

Inserção: A inserção de uma nova chave ocorre inicialmente em um nó folha. Caso o nó esteja cheio, realiza-se uma divisão (split), na qual a chave mediana é promovida para o nó pai e o conteúdo é distribuído entre dois novos nós. Esse processo mantém o balanceamento da estrutura e impede que os nós ultrapassem sua capacidade máxima.

Quando inserimos uma nova chave em uma árvore B , fazemos uma comparação da nova chave com as chaves que já estão na árvore, começando pela raiz, se a raiz possui duas chaves, comparamos com a primeira, se a nova chave for menor, ela “desce” para o nó filho à esquerda dessa chave, se for maior, comparamos com a segunda chave, se a nova chave for menor ela, faz o mesmo procedimento que fez com a primeira chave, e se a nova chave for maior que a última chave ela “desce” para o nó filho à direita da última chave, esse procedimento irá ocorrer até chegarmos em um nó folha onde a nova chave irá ser inserida.

O procedimento Dividir_No_Arvore_B toma como entrada um nó interno x não cheio e um índice i tal que $x.filhos[i]$ seja um nó cheio. Então esse procedimento reparte esse filho em dois e ajusta x de modo que ele tenha um filho adicional. Para repartir uma raiz cheia, primeiro transformamos a raiz em um filho de um novo nó raiz vazio para podermos usar Dividir_No_Arvore_B. Assim a altura da árvore aumenta em uma unidade, repartir é a única forma da árvore crescer. Como não podemos inserir uma

chave em um nó folha cheio, recorremos a uma operação que reparte em dois um nó cheio 'y' (que tem $2t-1$ chaves) em torno de sua chave mediana (uma chave que ordenada com as demais chaves do nó, está no meio da sequência) em dois nós que têm somente $t-1$ chaves. A chave mediana sobe para dentro do nó pai de y para identificar o ponto de repartição entre os dois novos nós.

O procedimento `Inserção_Na_Arvore_B` insere uma chave k em uma árvore B T utilizando o procedimento `Dividir_No_Arvore_B` para garantir que a recursão nunca desça em um nó cheio.

`Inserção_Na_Arvore_B(T, k)`

```

1      r = T.raiz
2      if r.n == 2t - 1
3          s = Alocar_No()
4          T.raiz = s
5          s.folha = false
6          s.n = 0
7          s.filhos[1] = r
8          Dividir_No_Arvore_B(s, 1)
9          Inserção_Nao_Cheio_Na_Arvore_B(s, k)
10     else Inserção_Nao_Cheio_Na_Arvore_B(r, k)

```

As linhas 3-9 tratam o caso no qual o nó de raiz r está cheio: a raiz é repartida e um novo nó s (que tem dois filhos) se torna a raiz. Repartir a raiz é o único modo de aumentar a altura de uma B-árvore, com isso a árvore só cresce para cima. O procedimento termina chamando `Inserção_Nao_Cheio_Na_Arvore_B` para inserir a chave k na árvore com raiz no nó de raiz não cheio.

A `Inserção_Nao_Cheio_Na_Arvore_B` executa recursão árvore abaixo conforme necessário e garante todas as vezes que o nó no qual executa a recursão não está cheio chamando `Dividir_No_Arvore_B` quando necessário.

O procedimento recursivo auxiliar `Inserção_Nao_Cheio_Na_Arvore_B` insere a chave k no nó x, suposto não cheio quando o procedimento é chamado.

`Inserção_Nao_Cheio_Na_Arvore_B(x, k)`

```

1      i = x.n
2      if x.folha
3          while i >= 1 e k < x.chave[ i ]
4              x.chave[ i + 1 ] = x.chave[ i ]
5              i = i - 1
6          x.chave[ i + 1 ] = k
7          x.n = x.n + 1
8          Disk-Write(x)

```

```

9      else while i >= 1 e k < x.chave[ i ]
10          i = i - 1
11          i = i + 1
12          Disk-Read(x.filhos[ i ])
13          if x.filhos[ i ].n = 2t - 1
14              Dividir_No_Arvore_B(x, i)
15              if k > x.chave[ i ]
16                  i = i + 1
17          Inserção_Nao_Cheio_Na_Arvore_B(x.filhos[ i ], k)

```

As linhas 3-8 tratam o caso no qual x é um nó folha inserindo a chave k em x . Se x não é um nó folha, devemos inserir k no nó folha adequado na subárvore com raiz no nó interno x . Nesse caso, as linhas 9-11 determinam o filho de x para o qual a recursão é descendente. A linha 13 detecta se a recursão necessária até um filho cheio, caso em que a linha 14 usa `Dividir_No_Arvore_B` para repartir esse filho em dois filhos não cheios, e as linhas 15-16 determinam qual dos dois filhos é agora o filho correto para o qual descer.

A complexidade da operação é de $O(t h) = O(t \log t n)$. h a altura da árvore, já que a operação de inserção vai até um nó folha, e t porque os laços `while` das linhas 3 e 9 se repetem no máximo $2t$ vezes.

À medida que descemos a árvore à procura da posição à qual pertence a nova chave, repartimos cada nó cheio que encontrarmos pelo caminho. Assim, sempre que queremos repartir um nó cheio ' y ', temos a certeza de que seu pai não está cheio.

`Dividir_No_Arvore_B(x, i)`

```

1      z = Alocar_No()
2      y = x.filho[ i ]
3      z.folha = y.folha
4      z.n = t - 1
5      for j = 1 to t - 1
6          z.chave[ j ] = y.chave[ j + t ]
7      if not y.folha
8          for j = 1 to t
9              z.filho[ j ] = y.filho[ j + t ]
10     y.n = t - 1
11     for j = x.n + 1 downto i + 1
12         x.filho[ j + 1 ] = x.filho[ j ]
13     x.filho[ i + 1 ] = z
14     for j = x.n downto i
15         x.chave[ j + 1 ] = x.chave[ j ]
16     x.chave[ i ] = y.chave[ t ]
17     x.n = x.n + 1
18     Disk-Write(y)

```

19 Disk-Write(z)

20 Disk-Write(x)

Remoção: A remoção é uma operação mais complexa, pois pode ocorrer tanto em folhas quanto em nós internos. Durante a exclusão, a árvore deve garantir que nenhum nó fique com menos chaves do que o permitido ($t - 1$). Para isso, podem ser realizadas redistribuições de chaves entre irmãos ou fusões de nós, preservando todas as propriedades da árvore B.

Visto que a maioria das chaves em uma B-árvore se encontra nas folhas, podemos esperar que, na prática, operações de eliminação são usadas na maior parte das vezes para eliminar chaves de folhas.

Instruções para remoção de uma chave:

1. Se a chave k está no nó x e x é uma folha, elimine a chave k de x .
2. Se a chave k está no nó x e x é um nó interno:
 - a. Se o filho y que precede k no nó x tem no mínimo t chaves, então encontre o predecessor k' (maior chave do nó y) de k na subárvore com raiz em y . Elimine recursivamente k' , e substitua k por k' em x .
 - b. Se y tiver menos que t chaves, então, simetricamente, examine o filho z que segue k no nó x . Se z tiver no mínimo t chaves, encontre o sucessor k' (menor chave do nó z) de k na subárvore com raiz em z . Elimine k' recursivamente e substitua k por k' em x .
 - c. Caso contrário, y e z têm apenas $t - 1$ chaves, junte k e todo o z com y , de modo que x perde k e também o ponteiro para z , e agora y contém $2t - 1$ chaves. Em seguida, libere z e elimine recursivamente k de y .
3. Se a chave k não estiver presente no nó interno x , determine a raiz $x.filho[i]$ da subárvore adequada que deve conter k , se k estiver na árvore. Se $x.filho[i]$ tiver somente $t - 1$ chaves, execute a etapa 3a ou 3b conforme necessário para garantir que descendermos até um nó que contém no mínimo t chaves. Então, termine executando recursão no filho adequado de x .
 - a. Se $x.filho[i]$ tiver somente $t - 1$ chaves, mas tiver um irmão imediato com no mínimo t chaves, dê a $x.filho[i]$ uma chave extra passando uma chave de x para baixo até $x.filho[i]$, passando uma chave do irmão imediato à esquerda ou à direita de $x.filho[i]$ para cima até x , e passando o ponteiro de filho adequado do irmão para $x.filho[i]$.
 - b. Se $x.filho[i]$ e os irmãos imediatos de $x.filho[i]$ têm $t - 1$ chaves, junte $x.filho[i]$ com um irmão, o que envolve passar uma chave de x para baixo até o novo nó resultante da junção, que assim se torna a chave mediana para esse nó.

Graças a essas características, as operações de busca, inserção e remoção possuem complexidade temporal logarítmica, tornando a estrutura altamente eficiente para grandes conjuntos de dados.

Exemplos de uso

Banco de dados: Uma das aplicações mais comuns das árvores B está nos Sistemas Gerenciadores de Banco de Dados (SGBDs). Elas são utilizadas na implementação de índices que permitem localizar registros rapidamente sem a necessidade de percorrer toda a tabela.

Por exemplo, ao realizar uma consulta pelo número de matrícula de um aluno ou pelo CPF de um cliente, o banco de dados pode utilizar uma árvore B para encontrar o registro desejado em poucos acessos ao disco, mesmo quando a base de dados contém milhões de registros.

Gerenciador de Arquivos: Muitos sistemas de arquivos utilizam árvores B ou suas variações para organizar diretórios e arquivos. Essa estrutura permite localizar rapidamente arquivos armazenados em disco e gerenciar grandes volumes de dados de forma eficiente.

Ao procurar um arquivo em uma pasta contendo milhares de documentos, o sistema operacional pode utilizar uma árvore B para encontrar a localização do arquivo sem precisar examinar todos os elementos presentes no diretório.

Exemplos de exercícios

1: Se a raiz de uma árvore B de grau mínimo 3 sofrer uma divisão (split), quantas chaves a nova raiz possuirá logo após a divisão?

R: Para um nó sofrer um split ele tem que estar completamente cheio, ou seja ter $2 * t - 1$ chaves em seu interior, como o grau mínimo dessa árvore é 3 a raiz terá que ter $2 * 3 - 1 = 5$ chaves em seu interior. O procedimento de split em uma raiz, por ela não ter um nó pai, cria um novo nó vazio que receberá a chave mediana da raiz, a antiga raiz será dividida em 2 nós com $t - 1$ chaves. logo a nova raiz terá 1 chave.

2: Determine qual é a altura máxima possível para uma árvore B que armazena $n = 50$ chaves, sabendo que seu grau mínimo é $t = 3$.

R: Como a altura h de uma árvore é $h \leq \log_2 \left(\frac{n+1}{2} \right)$. Substituindo os valores na fórmula, temos $h \leq \log_3 \left(\frac{50+1}{2} \right)$, $h \leq \log_3 (25,5)$, $h \leq 2,947$. Portanto a altura máxima da árvore de $n = 50$ e $t = 3$ é $h = 2$.

3: Em uma árvore B com $t = 3$, um nó interno possui o pai com a chave $[30, 70]$. Esse pai aponta para dois filhos: o filho esquerdo $A = [10, 20]$ e o filho direito $B = [40, 50, 60]$. O algoritmo solicita a remoção da chave 10 que está no nó A. Como a árvore se reestrutura?

R: Não podemos simplesmente remover a chave 10 do nó A porque ele já tem o número mínimo de chaves, já que $t = 3$ e o número mínimo é de $t - 1 = 2$. Então olhamos para o irmão de A, B tem 3 chaves, ou seja um a mais que o mínimo, então podemos pegar uma chave de B e fazer uma rotação de chaves. Na rotação passamos a chave 30 do nó pai para o nó A e passamos a chave 40 do nó B para o nó pai, então removemos a chave 10 do nó A. No final teremos o nó pai igual a $[40, 70]$, o nó A = $[20, 30]$ e o nó B = $[50, 60]$.

Referências

CORMEN, Thomas H. et al. Algoritmos: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

Árvore B. In: *Wikipédia: a enciclopédia livre*. [S. l.], 29 de janeiro de 2024 . Disponível em: [https://pt.wikipedia.org/wiki/%C3%81rvore_B].

OPENAI. *ChatGPT* (Versão 4.0). [S.l.]: OpenAI, 2026. Disponível em: <https://chatgpt.com>.